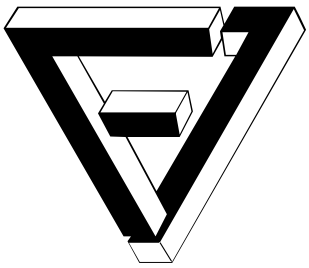
Masaryk University
Faculty of Informatics



Side-Effect Analysis Using a Static Single Use Program Representation

Master's Thesis

Samuel Malec

Brno, 2026

Declaration

Hereby I declare that this thesis is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used Claude Code (Anthropic, `https://claude.ai`) to write and review source code, to run and summarise experiments, and to help structure the text. After using this tool, I checked the content and accept full responsibility for it.

Supervisor: Petr Ročkai

Acknowledgements

Write your acknowledgements here.

Abstract

Write your abstract here.

Keywords

side-effect analysis, static single use, linear types, points-to analysis, ahead-of-time compilation, memory optimization, thread escape analysis, lock elision, GraalVM Native Image

Contents

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Objectives . . . 1
- 1.3 Contributions . . . 1
- 1.4 Structure of the Thesis . . . 1

2 Background 2
- 2.1 Side Effects and Their Analysis . . . 2
- 2.2 Program Representations . . . 2
- 2.3 Graal and Its Memory Model . . . 2
- 2.4 Native Image . . . 2
- 2.5 The Java Memory Model . . . 2

3 Analysis of the Problem 3
- 3.1 What a Call Costs the Optimizer . . . 3
- 3.2 What Information Would Help . . . 3
- 3.3 Requirements . . . 3
- 3.4 Approach . . . 3

4 Explicit Effects 4
- 4.1 A Language with Heap Tokens . . . 4
- 4.2 Semantics . . . 4
- 4.3 Partitions . . . 4
- 4.4 Signatures . . . 4
- 4.5 What the Tokens Guarantee . . . 4
- 4.6 Exceptions and Deoptimization . . . 4
- 4.7 Concurrency . . . 4
- 4.8 From Signatures to Kill Sets . . . 4

5 Implementation in GraalVM Native Image 5
- 5.1 Architecture . . . 5

5.2 Exporting the Points-to Analysis . . . 5
5.3 Translation to the Token Form . . . 5
5.4 Signatures and Kill Sets . . . 5
5.5 Back into Graal . . . 5
5.6 Thread Escape in Native Image's Analysis . . . 5
5.7 Lock Elision and Ordinary Volatile Accesses . . . 5

6 Evaluation 6
6.1 Method . . . 6
6.2 What Graal Optimizes with the Facts (RQ1) . . . 6
6.3 Cost against Benefit (RQ2) . . . 6
6.4 What Modelling Concurrency Enables (RQ3) . . . 6
6.5 Soundness in Practice (RQ4) . . . 6
6.6 What Blocks the Analysis . . . 6
6.7 Threats to Validity . . . 6

7 Related Work 7
7.1 Side-Effect and Mod/Ref Analyses . . . 7
7.2 Memory in Compiler Representations . . . 7
7.3 Escape Analysis and Synchronization Removal . . . 7
7.4 Analyses for Ahead-of-Time Compilation . . . 7

8 Conclusion 8
8.1 Results . . . 8
8.2 Contribution . . . 8
8.3 Future Work . . . 8

A Electronic Attachments 9

B Proofs 10

C The Dialect of Graal IR in Cthulhu 11

D Reproducing the Results 12

I Introduction

1.1 Motivation

1.2 Objectives

1.3 Contributions

1.4 Structure of the Thesis

2 Background

2.1 Side Effects and Their Analysis

2.2 Program Representations

2.3 Graal and Its Memory Model

2.4 Native Image

2.5 The Java Memory Model

3 Analysis of the Problem

3.1 What a Call Costs the Optimizer

3.2 What Information Would Help

3.3 Requirements

3.4 Approach

4 Explicit Effects

4.1 A Language with Heap Tokens

4.2 Semantics

4.3 Partitions

4.4 Signatures

4.5 What the Tokens Guarantee

4.6 Exceptions and Deoptimization

4.7 Concurrency

4.8 From Signatures to Kill Sets

5 Implementation in GraalVM Native Image

5.1 Architecture

5.2 Exporting the Points-to Analysis

5.3 Translation to the Token Form

5.4 Signatures and Kill Sets

5.5 Back into Graal

5.6 Thread Escape in Native Image's Analysis

5.7 Lock Elision and Ordinary Volatile Accesses

6 Evaluation

6.1 Method

6.2 What Graal Optimizes with the Facts (RQ1)

6.3 Cost against Benefit (RQ2)

6.4 What Modelling Concurrency Enables (RQ3)

6.5 Soundness in Practice (RQ4)

6.6 What Blocks the Analysis

6.7 Threats to Validity

7 Related Work

7.1 Side-Effect and Mod/Ref Analyses

7.2 Memory in Compiler Representations

7.3 Escape Analysis and Synchronization Removal

7.4 Analyses for Ahead-of-Time Compilation

8 Conclusion

8.1 Results

8.2 Contribution

8.3 Future Work

A Electronic Attachments

B Proofs

C The Dialect of Graal IR in Cthulhu

D Reproducing the Results